

Credit Card Default Prediction on Kaggle Dataset

Group 2

Hao Yang Qifan Zhou Zhesheng Xu
Yingqi Tan Cuijia Wei Shu Zhang

MISY262 presentation

December 15, 2025

Outline

- 1 Business Background & Data
- 2 Preprocessing & EDA
- 3 Modeling Approach
- 4 Model Results
- 5 Discussion & Conclusion
- 6 References

- Bank wants to predict whether a credit card client will **default next month**.
- This is a **binary classification** problem:
 - $Y = 1$: client defaults next month.
 - $Y = 0$: client does not default.
- From a financial mathematics perspective:
 - Model output can be interpreted as **probability of default (PD)**.
 - PD is a key input for **expected loss**:

$$EL = PD \times LGD \times EAD.$$

Research Question

Can we build accurate and interpretable models to predict credit card default on this Kaggle dataset, and how can a cost-sensitive decision rule improve risk management?

- Apply and compare five models: LR, Tree, RF, XGBoost, cost-sensitive ensemble.
- Interpret results from a financial mathematics perspective (PD, EL).
- Propose a simple cost-sensitive ensemble to reflect asymmetric misclassification costs.

Dataset Description

- Dataset: Kaggle / UCI *Default of Credit Card Clients*.
- Number of observations:
 - Training set: **21,001** rows.
 - Test set: **8,999** rows.
- Variable groups:
 - Demographics: SEX, AGE, EDUCATION, MARRIAGE.
 - Credit and bill info: LIMIT_BAL, BILL_AMT1--6, PAY_AMT1--6.
 - Repayment status: PAY_0--PAY_6.
 - Target: default_next_month.
- Default rate in the test set is around **22%**.

Data Preprocessing

- Drop identifier ID.
- Rename target:

`default.payment.next.month` $\rightarrow$ `default_next_month`.

- Recode target as factor:

`0` $\rightarrow$ `no_default`, `1` $\rightarrow$ `default`.

- Clean and recode categorical variables:
 - EDUCATION: codes 0,4,5,6 grouped as “others”.
 - MARRIAGE: code 0 grouped as “others”.
- Convert SEX, EDUCATION, MARRIAGE, PAY_0--PAY_6 to factors.
- Split data into **70% training** and **30% testing** with stratified sampling.

Missing Data and Imputation

- Checked missing values using:
 - `sum(is.na()), colSums(is.na())`.
 - Visual tools such as `vis_miss()`, `gg_miss_var()`.
- Result:
 - This dataset contains **no missing values**.
 - Therefore, no imputation is required in practice.
- We still documented a general framework based on `mice` for datasets with missing values.

Exploratory Data Analysis (Examples)

- Distribution of credit limit `LIMIT_BAL`:
 - Highly right-skewed.
 - Most clients have low to medium credit limits.
- Default rates by categorical variables:
 - Higher default rates for certain education and repayment status groups.
- Correlation analysis:
 - Strong correlation among bill amounts and payment amounts.
 - Important when interpreting logistic regression coefficients.

Train/Test Split and Design

- Training set used for model estimation and tuning.
- Test set used only for final performance evaluation.
- For XGBoost:
 - Removed rows with NA values (`na.omit`) to form `train_xgb`, `test_xgb`.
 - Used `model.matrix` to create one-hot encoded design matrices.
- Ensured factor levels are consistent between training and test data.

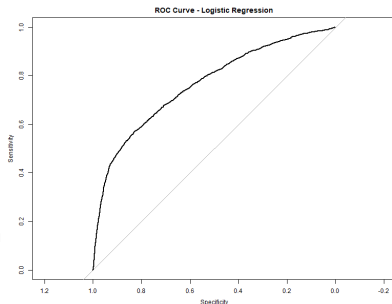
Models Compared

- **Model 1:** Logistic Regression (baseline PD model).
- **Model 2:** Decision Tree (rpart).
- **Model 3:** Random Forest.
- **Model 4:** XGBoost.
- **Model 5:** Cost-sensitive Ensemble:
 - Average probabilities from logistic regression, random forest, and XGBoost.
 - Choose classification threshold to minimise expected cost of errors.

- Metrics from `caret::confusionMatrix`:
 - Accuracy.
 - Sensitivity (recall for default class).
 - Specificity.
 - Balanced Accuracy.
- Discrimination:
 - **ROC curve** and **AUC** (Area Under the Curve).
 - Higher AUC = better ability to rank risky clients above safe ones.

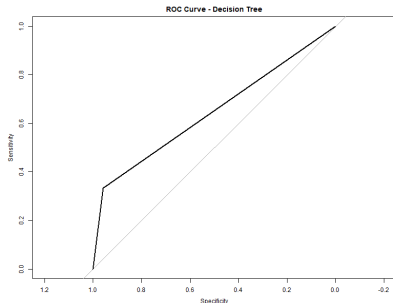
Model 1: Logistic Regression

- Standard logistic regression with all predictors.
- Test-set performance:
 - Balanced Accuracy: **0.6552**.
 - AUC: **0.7631**.
- Provides interpretable coefficients and a natural PD estimate.



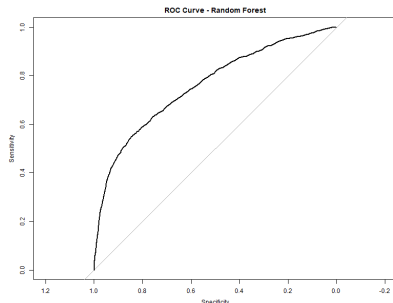
Model 2: Decision Tree

- Simple and interpretable recursive partitioning.
- Test-set performance:
 - Accuracy: **0.8202**.
 - Sensitivity: **0.3327**.
 - Specificity: **0.9586**.
 - Balanced Accuracy: **0.6456**.
 - AUC: **0.6456**.
- Very high specificity but low sensitivity: many defaulters are missed.



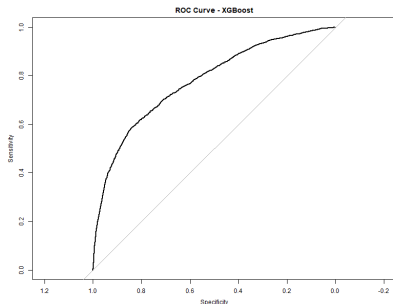
Model 3: Random Forest

- Ensemble of many decision trees with bootstrap sampling and random features.
- Test-set performance:
 - Accuracy: **0.8195**.
 - Sensitivity: **0.3851**.
 - Specificity: **0.9428**.
 - Balanced Accuracy: **0.6640**.
 - AUC: **0.7604**.
- Improves sensitivity relative to a single tree, with strong overall performance.



Model 4: XGBoost

- Gradient boosting framework optimized for tabular data.
- Test-set performance (NA-free test set):
 - Accuracy: **0.8195**.
 - Sensitivity: **0.3736**.
 - Specificity: **0.9461**.
 - Balanced Accuracy: **0.6598**.
 - AUC: **0.7777**.
- Highest AUC among single models, strong ranking power.



Model 5: Cost-Sensitive Ensemble

- Ensemble probability:

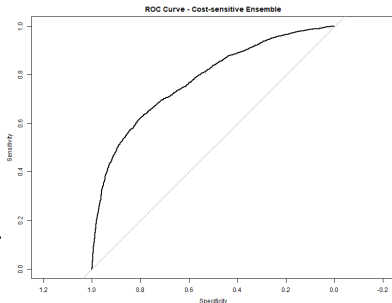
$$\hat{p}_{\text{ens}} = \frac{\hat{p}_{\text{logit}} + \hat{p}_{\text{rf}} + \hat{p}_{\text{xgb}}}{3}.$$

- Choose threshold t to minimize cost:

$$\text{Cost} = C_{\text{FN}} \cdot \text{FN} + C_{\text{FP}} \cdot \text{FP}, \quad C_{\text{FN}} \gg C_{\text{FP}}.$$

- Best threshold found: $t = 0.10$.
- Test-set performance:

- Accuracy: **0.4283**.
- Sensitivity: **0.9397**.
- Specificity: **0.2831**.
- Balanced Accuracy: **0.6114**.
- AUC: **0.7780**.



Model Comparison Summary

Model	Accuracy	Bal. Acc.	AUC
Logistic Reg.	–	0.6552	0.7631
Decision Tree	0.8202	0.6456	0.6456
Random Forest	0.8195	0.6640	0.7604
XGBoost	0.8195	0.6598	0.7777
Ensemble	0.4283	0.6114	0.7780

- For balanced performance: Random Forest and XGBoost are strong choices.
- For very conservative risk management: the ensemble achieves extremely high sensitivity.

- Each model outputs a probability of default (PD) for every client.
- PD is combined with LGD and EAD to compute expected loss.
- Bank can:
 - Set different PD thresholds depending on risk appetite.
 - Adjust credit limits and interest rates.
 - Use cost-sensitive ensemble in scenarios where missing a default is extremely expensive.

Conclusion

- Built a full EDA and modeling pipeline on a Kaggle credit card default dataset.
- Compared five models:
 - Logistic Regression, Decision Tree, Random Forest, XGBoost, Cost-sensitive Ensemble.
- XGBoost and Random Forest provide strong predictive performance ($AUC \approx 0.76$ – 0.78).
- Cost-sensitive ensemble dramatically increases sensitivity, suitable for very risk-averse settings.
- Future work:
 - Hyperparameter tuning and feature engineering.
 - Calibration of predicted PDs.
 - Extension to other credit products and time periods.

References

- ① Breiman, L. "Random Forests," *Machine Learning* (45:1), 2001, pp. 5–32.
- ② Chen, T. "XGBoost: A Scalable Tree Boosting System," Cornell University, 2016.
- ③ Friedman, J.H. "Greedy Function Approximation: A Gradient Boosting Machine," *The Annals of Statistics* (29:5), 2001, pp. 1189–1232.
- ④ Hand, D.J., and Henley, W.E. "Statistical Classification Methods in Consumer Credit Scoring: A Review," *Journal of the Royal Statistical Society: Series A (Statistics in Society)* (160:3), 1997, pp. 523–541.
- ⑤ Thomas, L.C. *Consumer Credit Models: Pricing, Profit and Portfolios*, OUP Oxford, 2009.

Thank you!

Questions?